LAB # 07

Exercise:

1. Validate Basic Calculator application through unitest module.use atleast 6 assertions methods in your testfile. //Attach all code snippets and ouputs here.

Calculator Code in Python:

calculator.py ● unit_test.py ●

calculator.py > ...

```python
import tkinter as tk
from tkinter import messagebox
import calculator

def add(x, y):
    return x + y
def subtract(x, y):
    return x - y
def multiply(x, y):
    return x * y
def divide(x, y):
    if y == 0:
        raise ValueError("Cannot divide by zero")
    return x / y

def calculate(operation):
    try:
        a = float(entry1.get())
        b = float(entry2.get())

        if operation == "Add":
            result = calculator.add(a, b)
        elif operation == "Subtract":
            result = calculator.subtract(a, b)
        elif operation == "Multiply":
            result = calculator.multiply(a, b)
        elif operation == "Divide":
            result = calculator.divide(a, b)
        else:
```

```
            else:
                result = "Invalid Operation"

            result_label.config(text=f"Result: {result}")
        except Exception as e:
            messagebox.showerror("Error", str(e))

root = tk.Tk()
root.title("Basic Calculator")
root.geometry("500x400")
root.resizable(True, True)
root.config(bg="lightyellow")

# Title Label
title_label = tk.Label(root, text="Basic Calculator", font=("Arial", 16), bg="lightyellow",
fg="darkblue")
title_label.pack(pady=20)

tk.Label(root, text="Enter first number:", bg="lightyellow", font=("Arial", 12)).pack(pady=5)
entry1 = tk.Entry(root, font=("Arial", 12), width=20)
entry1.pack(pady=5)

tk.Label(root, text="Enter second number:", bg="lightyellow", font=("Arial", 12)).pack(pady=5)
entry2 = tk.Entry(root, font=("Arial", 12), width=20)
entry2.pack(pady=5)

button_frame = tk.Frame(root, bg="lightyellow")
button_frame.pack(pady=10)
```

```

operations = ["Add", "Subtract", "Multiply", "Divide"]
for operation in operations:
    button = tk.Button(button_frame, text=operation, command=lambda op=operation: calculate(op),
    width=12, font=("Arial", 12), bg="lightblue", relief="solid")
    button.pack(side=tk.LEFT, padx=5)

calculate_button = tk.Button(root, text="Calculate", command=lambda: calculate("Custom"),
width=20, font=("Arial", 12), bg="lightgreen", relief="solid")
calculate_button.pack(pady=15)


result_label = tk.Label(root, text="Result: ", bg="lightyellow", font=("Arial", 12), fg="darkgreen")
result_label.pack(pady=10)

root.mainloop()

```

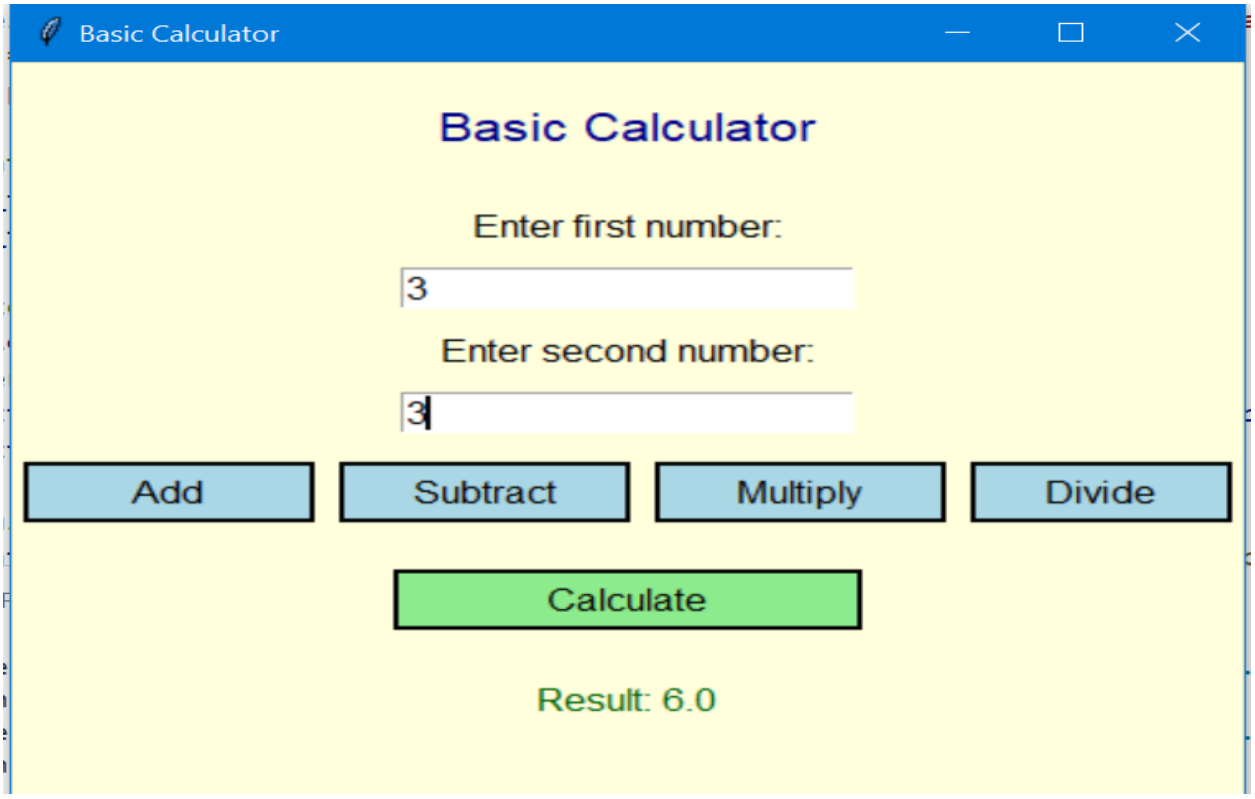
Unit Test Code:

calculator.py | unit_test.py ●

unit_test.py > ...

```python
import unittest
import calculator

class TestCalculator(unittest.TestCase):

    def test_add(self):
        self.assertEqual(calculator.add(10, 5), 15)

    def test_subtract(self):
        self.assertNotEqual(calculator.subtract(10, 5), 10)

    def test_multiply(self):
        self.assertEqual(calculator.multiply(3, 7), 21)

    def test_divide(self):
        self.assertAlmostEqual(calculator.divide(10, 3), 3.333333, places=4)

    def test_divide_by_zero(self):
        with self.assertRaises(ValueError):
            calculator.divide(5, 0)

    def test_type(self):
        self.assertIsInstance(calculator.add(1, 2), int)

if __name__ == '__main__':
    unittest.main()

```

Output:

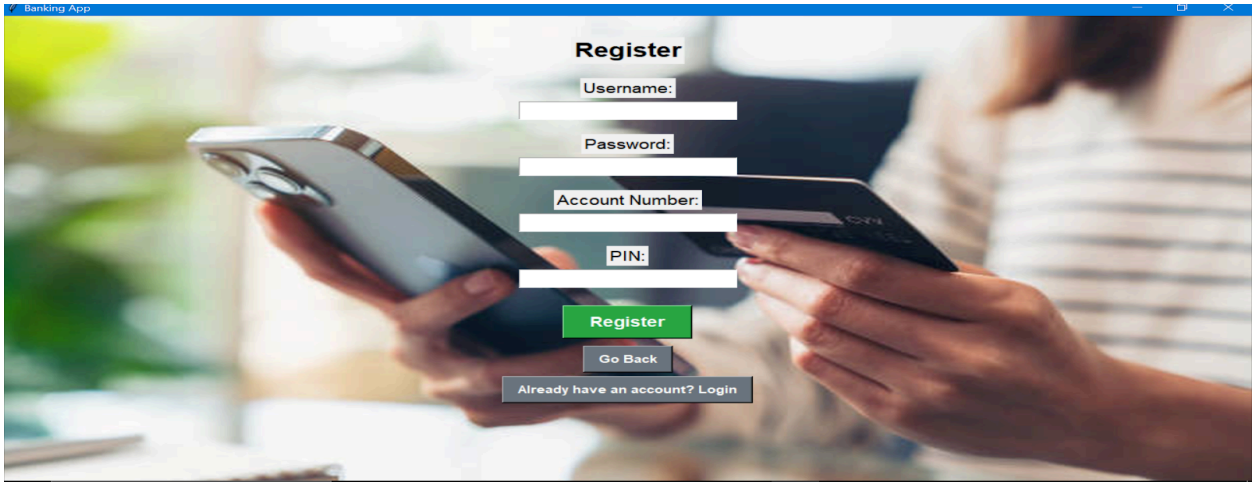



```
PS C:\Users\lenovo\Desktop\calculator_python> & "C:/Program Files/Python313/python.exe" c:/Users/lenovo/Desktop/c
alculator_python/unit_test.py
......
----------------------------------------------------------------------
Ran 6 tests in 0.009s

OK
```

2. Validate any 2 module of your previous python application through unitest //. //Attach all code snippets and ouputs here.

REGISTRATION MODULE:

MODULE UI:



PYTHON CODE :

reg.py > ...

```python
# register.py

class User:
    def __init__(self, username, password, balance=0):
        self.username = username
        self.password = password
        self.balance = balance

class UserRegistry:
    def __init__(self):
        self.users = {}

    def register(self, username, password):
        if username in self.users:
            return "User already exists"
        if not username or not password:
            return "Username and password are required"
        self.users[username] = User(username, password)
        return "Registration successful"

    def get_user(self, username):
        return self.users.get(username)
```

UNIT TEST CODE :

```python
import unittest
from reg import UserRegistry

class TestUserRegistration(unittest.TestCase):

    def setUp(self):
        self.registry = UserRegistry()

    def test_register_successful(self):
        result = self.registry.register("john", "pass123")
        self.assertEqual(result, "Registration successful")

    def test_register_duplicate_user(self):
        self.registry.register("john", "pass123")
        result = self.registry.register("john", "newpass")
        self.assertEqual(result, "User already exists")

    def test_register_empty_username(self):
        result = self.registry.register("", "pass123")
        self.assertEqual(result, "Username and password are required")

    def test_register_empty_password(self):
        result = self.registry.register("john", "")
        self.assertEqual(result, "Username and password are required")

    def test_get_user_after_registration(self):
        self.registry.register("john", "pass123")
        user = self.registry.get_user("john")
        self.assertIsNotNone(user)
        self.assertEqual(user.username, "john")
        self.assertEqual(user.password, "pass123")

if __name__ == "__main__":
    unittest.main()
```

OUTPUT :

```
[Running] python -u "c:\Users\lenovo\Desktop\BankingSystem-Test\reg.py"

[Done] exited with code=0 in 0.283 seconds

[Running] python -u "c:\Users\lenovo\Desktop\BankingSystem-Test\reg-unittest.py"
.....
----------------------------------------------------------------------
Ran 5 tests in 0.000s
```

WITHDRAW MODULE:

MODULE UI:


Welcome, SHEHNILA
Balance: $0
Deposit
Deposit
Withdraw
Withdraw
Recipient's Name
Transfer
Logout

PYTHON CODE ;

withdraw.py > ...

```python
# withdraw.py

class User:
    def __init__(self, username, password, balance=0):
        self.username = username
        self.password = password
        self.balance = balance

class BankSystem:
    def __init__(self):
        self.users = {}

    def register_user(self, username, password, balance=0):
        if username not in self.users:
            self.users[username] = User(username, password, balance)
            return "User registered"
        return "User already exists"

    def withdraw(self, username, amount):
        user = self.users.get(username)
        if not user:
            return "User not found"
        if amount <= 0:
            return "Invalid amount"
        if user.balance < amount:
            return "Insufficient balance"
        user.balance -= amount
        return f"Withdrawn {amount} successfully"

    def get_balance(self, username):
        user = self.users.get(username)
        return user.balance if user else None
```

UNIT TEST CODE :

withdraw-unittest.py > TestWithdraw > test_withdraw_insufficient_balance

```python
# test_withdraw.py

import unittest
from withdraw import BankSystem

class TestWithdraw(unittest.TestCase):

    def setUp(self):
        self.bank = BankSystem()
        self.bank.register_user("john", "pass123", 1000)

    def test_successful_withdrawal(self):
        result = self.bank.withdraw("john", 500)
        self.assertEqual(result, "Withdrawn 500 successfully")
        self.assertEqual(self.bank.get_balance("john"), 500)

    def test_withdraw_insufficient_balance(self):
        result = self.bank.withdraw("john", 1500)
        self.assertEqual(result, "Insufficient balance")
        self.assertEqual(self.bank.get_balance("john"), 1000)

    def test_withdraw_invalid_user(self):
        result = self.bank.withdraw("jane", 100)
        self.assertEqual(result, "User not found")

    def test_withdraw_invalid_amount(self):
        result = self.bank.withdraw("john", -50)
        self.assertEqual(result, "Invalid amount")

if __name__ == "__main__":
    unittest.main()

```

OUTPUT :

```
[Running] python -u "c:\Users\lenovo\Desktop\BankingSystem-Test\withdraw-unittest.py"
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```